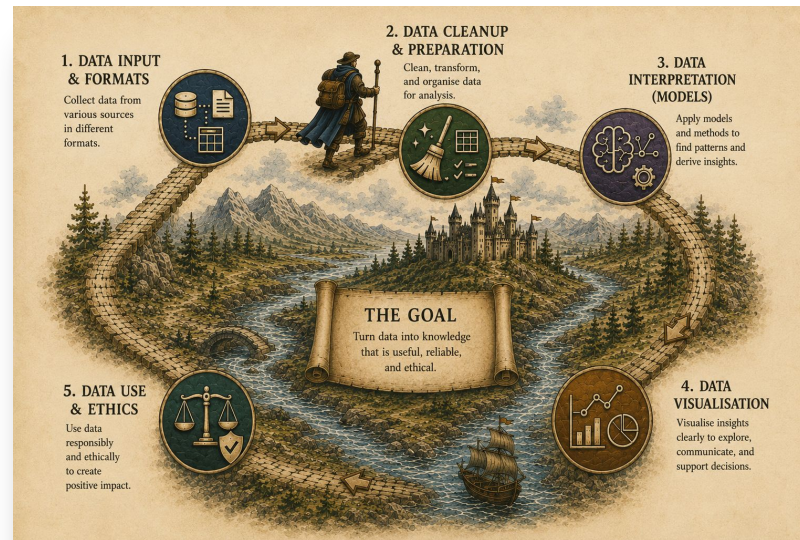


COMP20008 Elements of Data Processing



Lecture 6: Text Processing (“text normalisation and pattern matching”)

Semester 2 2026

Subject Coordinator: **Dr Ekaterina Vylomova**

Co-lecturer: **Dr Hanzhuo (Vivian) Ma**



THE UNIVERSITY OF
MELBOURNE

A1: Released!

AirBNB for Melbourne!

	A	B	C	D	E	F	G	H	I	J											AC						
id	name	description	host_id	host_is_su	host_listin	neighbour	latitude	longitude	property_i	room_tpy	accommod	bathroom:	bedrooms	beds	amenities	price	minimum	maximum	availability	has_availa	number_o	first_revie	last_revie	review_sci	review_sci	review_sci	reviews_per
2	10803	Room in D A large air	38901	t	f	1 Moreland	-37.7661	144.9795	Private roc	Private roc	2	1 shared bath			1 ["Self chec	\$60.74	5	28	158 t	214	#####	4/5/2026	4.57	4.07	4.66	1.31	
3	12936	St Kilda 18 RIG IN T	50121	t	f	10 Port Phillip	-37.86	144.9766	Entire rent	Entire hon	2	1 bath		1	["Dishes and silverwa		3	14	0 t	42	4/8/2010	15/3/2026	4.68	4.71	4.66	0.22	
4	41836	CLOSE TO Easy to tra	182833	t	f	3 Darebin	-37.6976	145.0007	Private roc	Private roc	2	1 shared bath			1 ["Heating", "Hair drye		7	1125	0 t	157	16/11/2012	22/8/2018	4.71	4.65	4.69	0.83	
5	43429	Tranquil Ja No service	189684	t	t	4 Monash	-37.8998	145.1158	Entire rent	Entire hon	2	1 bath		1	1 ["Self chec	\$165.00	2	1125	278 t	280	#####	14/5/2026	4.88	4.94	4.86	1.48	
6	44699	15 yearsHi Unwaverir	189245	t	f	3 Port Phillip	-37.8316	144.9706	Private roc	Private roc	2	1 shared bath			2 ["Bay view	\$215.00	1	14	169 t	67	9/9/2010	17/5/2026	4.77	4.83	4.71	0.35	
7	47100	Cosy, cute Our home	212071	t	f	2 Yarra	-37.8184	145.005	Entire rent	Entire hon	2	1 bath		1	1 ["Long term stays allc		4	21	0 t	180	9/9/2010	8/1/2025	4.88	4.92	4.81	0.94	
8	51592	Central Cit Stay in a st	190879	t	t	2 Melbourne	-37.8127	144.9631	Entire loft	Entire hon	2	1.5 baths		1	1 ["Self chec	\$294.10	2	1125	208 t	537	22/2/2011	7/6/2026	4.91	4.75	4.79	2.88	
9	2067618	Homely 3- Feel at hor	10575274	t	f	1 Whitehors	-37.8276	145.1175	Entire tow	Entire hon	6	2 baths		3	3 ["Self chec	\$207.50	2	365	77 t	13	#####	13/6/2026	5	5	5	2.02	
10	2079904	Modern 1-â"™... Stunn	10394006	t	t	14 Melbourne	-37.8166	144.9533	Entire rent	Entire hon	4	1 bath		2	2 ["Sauna", "Elevator"	\$192.12	2	1125	46 t	374	6/1/2014	#####	4.83	4.89	4.7	2.47	
11	2084207	Apartmen With its m	976168	t	f	1 Yarra	-37.8046	144.9832	Entire rent	Entire hon	2	1 bath		1	1 ["Heating", "Elevato		3	30	0 t	10	14/2/2016	16/7/2017	4.9	4.6	4.56	0.08	
12	2099179	Reduced-r Cat lover r	2831931	t	f	1 Port Phillip	-37.859	144.9984	Entire rent	Entire hon	2	1 bath		1	1 ["Self chec	\$85.71	7	21	22 t	100	26/1/2014	7/1/2026	4.96	4.97	4.93	0.66	
13	2106830	Wahroong I have ope	10741024	t	f	1 Melbourne	-37.7962	144.9684	Entire hon	Entire hon	6	1.5 baths		3	1 ["Long term stays allc		6	1125	0 t	178	13/8/2014	#####	4.93	4.87	4.81	1.23	
14	2119988	Sleep me i Great Spa	10810140	t	t	3 Melbourne	-37.8093	144.9598	Entire rent	Entire hon	2	1 bath		1	1 ["Self chec	\$175.00	2	1125	284 t	563	#####	7/6/2026	4.96	4.92	4.9	4.33	
15	2124756	Large One Melbourne	10836093	t	f	1 Port Phillip	-37.855	144.9815	Entire rent	Entire hon	2	1 bath		1	1 ["Long term stays allc		1	1125	0 t	34	18/3/2014	22/7/2023	4.79	4.76	4.73	0.23	
16	2128253	Sensation! Perfectly s	8908174	t	f	1 Port Phillip	-37.8626	144.9746	Entire rent	Entire hon	2	1 bath		1	1 ["Long term stays allc		3	200	0 t	30	3/1/2016	25/11/201	4.97	5	4.93	0.23	
17	2136008	Stylish Wa Welcome i	1444621	t	f	1 Yarra	-37.8035	144.9858	Entire rent	Entire hon	2	1.5 baths		1	1 ["Self chec	\$346.97	2	100	52 t	165	18/1/2014	#####	4.94	4.95	4.91	1.09	
18	2142770	â"™ Award This fully s	10936267	t	f	1 Port Phillip	-37.863	144.9877	Entire gue	Entire hon	4	1 bath			1 ["Outlet covers", "Iro		2	1125	0 t	1	20/1/2014	20/1/2014	5	5	5	0.01	
19	2150936	Queen bee Walking di	10857350	t	f	5 Wyndham	-37.8926	144.6678	Private roc	Private roc	2	2 baths			3 ["Self chec	\$87.00	1	180	259 t	73	13/4/2014	28/5/2026	4.71	4.85	4.74	0.49	
20	2156314	Spacious - Stay in you	3007990	t	f	2 Port Phillip	-37.8582	144.9693	Entire rent	Entire hon	4	1 bath		1	1 ["Self chec	\$240.00	3	730	220 t	85	24/1/2014	9/4/2026	4.93	4.8	4.86	0.56	
21	2157135	Quiet, clez Convenien	11008407	t	f	1 Stonningtc	-37.8524	145.0103	Private roc	Private roc	1	1 shared bath			1 ["First aid	\$80.80	5	31	11 t	32	20/10/2013	30/4/2026	4.88	4.94	4.77	0.23	
22	2178434	Large Mell Looking fo	11111107	t	t	1 Maribyrnc	-37.807	144.9019	Entire hon	Entire hon	15	3 baths		9	12 ["Self chec	\$953.00	3	1125	242 t	160	24/2/2014	19/4/2026	4.84	4.75	4.77	1.07	
23	2194291	Lovely Bric in the mid	11191682	t	f	1 Yarra Rang	-37.7505	145.3117	Private roc	Private roc	1	1 private b		1	1 ["Garden v	\$81.00	2	6	89 t	15	30/12/201	7/4/2024	4.2	4.13	4.53	0.11	
24	2234455	BEACHSID Modern aj	3013970	t	f	1 Bayside	-37.933	144.9999	Entire rent	Entire hon	2	1.5 baths		1	1 ["Long term stays allc		7	1125	0 t	42	29/4/2014	16/12/201	4.83	4.71	4.69	0.28	
25	2234483	Abbotsford's All-roun	11377202	t	f	1 Yarra	-37.803	144.9951	Entire hon	Entire hon	4	1 bath		2	1 ["Heating", "Washer"		2	1125	0 t	11	28/1/2014	24/11/201	5	5	5	0.07	
26	2238738	Nook Mell Welcome i	9510598	t	t	56 Melbourne	-37.8244	144.9603	Entire rent	Entire hon	6	2 baths		2	2 ["Self chec	\$292.67	2	365	96 t	280	7/2/2014	1/5/2026	4.72	4.71	4.73	1.86	
27	2245554	Beautiful (WELCOMI	11468856	t	f	1 Melbourne	-37.818	144.9875	Private roc	Private roc	2	1 private bath			1 ["Self chec	\$86.33	4	1125	43 t	316	6/2/2014	26/4/2026	4.71	4.73	4.76	2.1	
28	2258577	Art Deco q Our beaut	2657687	t	f	2 Port Phillip	-37.8629	144.9808	Entire rent	Entire hon	4	1 bath		2	1 ["Heating", "Smoke a		2	5	0 t	15	13/3/2014	17/3/2015	4.57	4.38	4.46	0.1	
29	2263264	Light & Sp Our light f	11557695	t	f	1 Port Phillip	-37.8823	144.9794	Entire rent	Entire hon	4	1 bath		2	1 ["Heating", "Hair drye		2	14	0 t	7	30/12/2012	24/4/2016	4.57	4.57	4.57	0.05	
30	2292052	Luxury Ma Converted	10621363	t	t	3 Port Phillip	-37.8599	144.9751	Entire loft	Entire hon	2	1 bath		1	1 ["Self chec	\$253.60	1	1125	199 t	531	15/5/2014	16/6/2026	4.79	4.84	4.74	3.61	
31	2301999	Hush â"™ Hi The Hush	11755700	t	t	5 Yarra Rang	-37.862	145.3595	Entire cott	Entire hon	2	1 bath		1	1 ["Self chec	\$480.00	1	20	342 t	264	#####	8/5/2026	4.55	4.68	4.53	1.85	
32	2357769	An extrad Exterior w	12041542	t	f	1 Port Phillip	-37.8618	144.9815	Entire hon	Entire hon	4	1.5 baths		2	2 ["Shared b	\$571.00	2	1125	314 t	63	22/4/2014	23/1/2026	4.73	4.43	4.74	0.43	
33	2361740	Stunning, Gorgeous	638404	t	f	1 Yarra	-37.8032	144.9843	Entire rent	Entire hon	4	1 bath		2	2 ["TV with	\$293.40	1	1125	165 t	60	19/8/2014	8/3/2026	4.97	4.92	4.93	0.42	
34	2362450	Renovated Recently r	10985248	t	f	1 Glen Eira	-37.8901	145.0662	Entire hon	Entire hon	6	2 baths		3	1 ["Iron", "Shampoo",		4	1125	0 t	4	13/5/2014	30/3/2015	4.75	5	4.5	0.03	
35	2374619	West Melt Self contai	12132807	t	f	1 Melbourne	-37.8051	144.947	Entire rent	Entire hon	1	1 bath		1	1 ["Heating", "Elevato		2	14	0 t	15	15/3/2014	18/1/2016	4.67	4.67	4.47	0.1	
36	2398060	2BR Rock Step into	12197796	t	t	12 Melbourne	-37.8162	144.9845	Entire rent	Entire hon	5	1 bath		2	5 ["Self chec	\$344.00	1	1125	114 t	462	22/4/2014	9/6/2026	4.68	4.6	4.69	3.12	
37	2405220	Melbourne PrivateAp	12293277	t	t	1 Moreland	-37.7755	144.9486	Entire rent	Entire hon	2	1 bath			1 ["Long teri	\$52.14	56	1125	208 t	71	#####	16/5/2026	4.76	4.87	4.74	0.54	
38	2413411	Quiet, yet Key's Plac	12298592	t	f	1 Melbourne	-37.7912	144.9298	Private roc	Private roc	2	1 private b		2	2 ["TV with	\$161.50	2	1125	359 t	129	19/5/2014	14/6/2026	4.8	4.61	4.83	0.88	
39	2419713	Lovingly re Set in com	12336757	t	f	4 Yarra Rang	-37.6059	145.4375	Entire cott	Entire hon	2	1.5 baths		1	1 ["Private e	\$567.00	2	1125	317 t	40	#####	23/11/202	4.98	5	4.9	0.27	
40	2426943	Great Locc Large mod	566559	t	f	1 Glen Eira	-37.8975	145.0091	Private roc	Private roc	4	3 baths		2	1 ["Heating", "Smoke a		2	1125	0 t	14	14/3/2014	2/4/2015	4.93	4.86	4.79	0.09	
41	2427366	180* View BUBBLING	8354765	t	f	2 Yarra Rang	-37.8185	145.3628	Entire rent	Entire hon	2	1 bath		2	3 ["TV with	\$400.00	1	7	305 t	175	20/4/2015	31/5/2026	4.71	4.66	4.46	1.29	
42	2427388	MAGNIFIC BUBBLING	8354765	t	f	2 Yarra Rang	-37.8194	145.3643	Entire rent	Entire hon	2	1 bath		1	1 ["Bed line	\$400.00	1	1125	295 t	248	#####	15/6/2026	4.81	4.73	4.58	1.75	
43	2428212	Flinders Ba Situated ir	12402075	t	t	4 Melbourne	-37.8165	144.9654	Entire rent	Entire hon	3	1 bath		1	2 ["Central h	\$251.00	2	730	331 t	202	19/5/2014	9/6/2026	4.84	4.86	4.82	1.37	
44	2432350	Victorian 2 Beautifully	12132450	t	f	1 Yarra	-37.7912	144.9893	Entire hon	Entire hon	4	1 bath		2	2 ["Self chec	\$374.71	7	30	173 t	38	29/6/2015	13/3/2026	4.68	4.58	4.66	0.28	
45	2448043	The Green Beautifully	12519742	t	f	1 Nillumbik	-37.6578	145.1891	Entire cott	Entire hon	5	1.5 baths		2	2 ["Self chec	\$269.00	1	730	341 t	53	6/1/2017	5/4/2026	4.89	4.94	4.75	0.46	
46	2449576	Southbank Our cool v	373049	t	f	1 Port Phillip	-37.8301	144.9588	Entire loft	Entire hon	4	1 bath		1	1 ["Dishes and silverwa		1	31	0 t	640	15/2/2015	23/11/202	4.91	4.91	4.91	4.62	
47	2478171	Stunning i queen bi	11432904	t	f	1 Maribyrnc	-37.7948	144.8839	Private roc	Private roc	2	1.5 baths			1 ["Heating", "Hair drye		1	352	0 t	16	15/9/2014	13/12/201	4.75	4.88	4.69	0.11	

A1: Released!

\$571 An extraordinary St Kilda mansion

Exterior walls feature crumbling pink paint, with an orange grove to greet you in the front garden. Beautifully and simply restored it has a French sensibility inside and an expansive garden. 4m high ceilings downstairs and upstairs. Rich with art and unique furnishings it has a glorious ambience inside

\$52 Melbourne, Brunswick, Aptmnt, CityViews

PrivateApartment. Medium to long stays. Bathroom & kitchen. Suits work from home. Brunswick is North of the city, but very close to the CBD. Light filled FULLY FURNISHED apartment in lovely building, includes linen. Tram and bus stops to hospitals and universities at front of building. Nearby: grocery, pharmacy, wine, nuts, bakery, restaurants, and take-aways. 200gb data refresh monthly on 27th then Guest pay for own WIFI



Recommend: Jurafsky's

[“The language of food”](#)

A1: Released!

\$571

An extraordinary St Kilda mansion

Exterior walls feature crumbling pink paint, with an orange grove to greet you in the front garden.

Beautifully and simply restored it has a French sensibility inside and an expansive garden.

4m high ceilings downstairs and upstairs.

Rich with art and unique furnishings it has a glorious ambience inside



A1: Released!

\$52

Melbourne, Brunswick,
Aptmnt, CityViews



Private Apartment.
Medium to long stays.



Bathroom & kitchen.
Suits work from home.



Brunswick is North of the city,
but very close to the CBD.



Light filled
FULLY FURNISHED
apartment in lovely building,
includes linen.



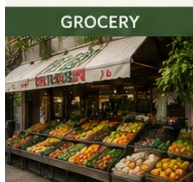
Tram and bus stops to
hospitals and universities
at front of building.



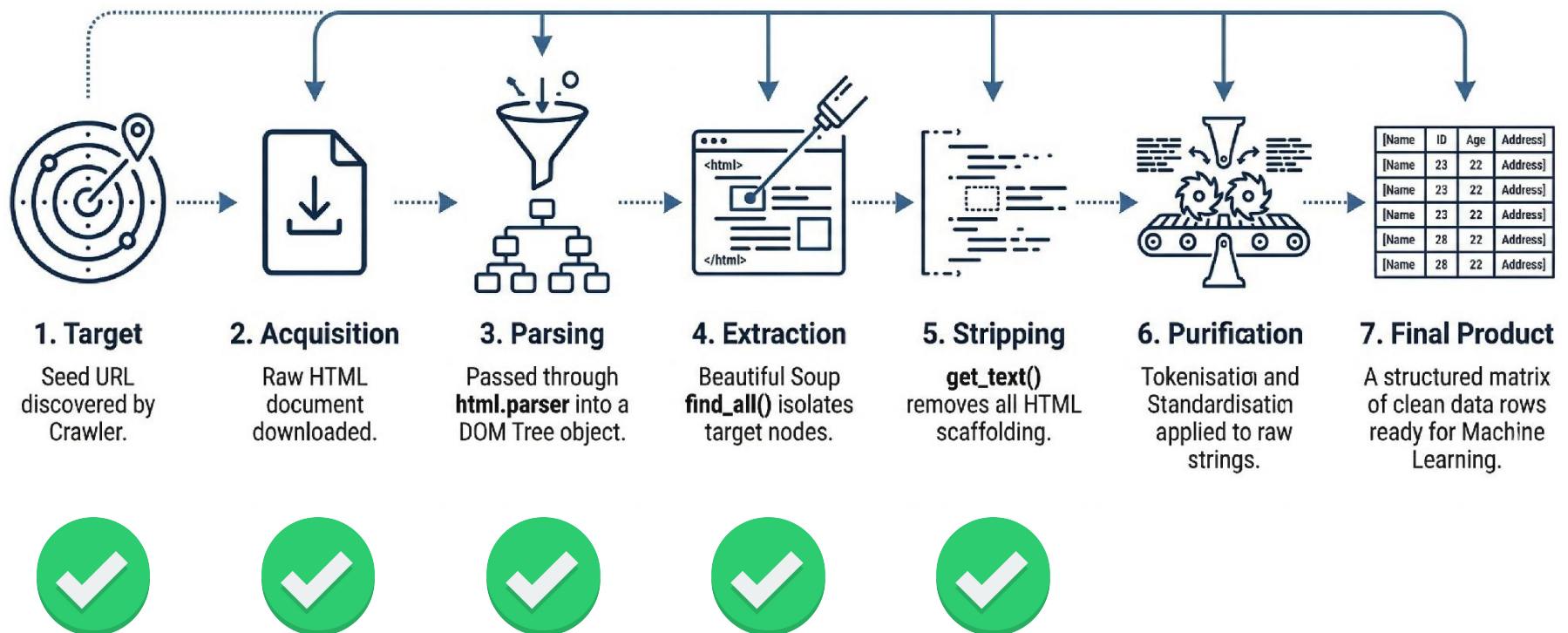
Nearby: grocery, pharmacy,
wine, nuts, bakery, restaurants,
and take-aways.



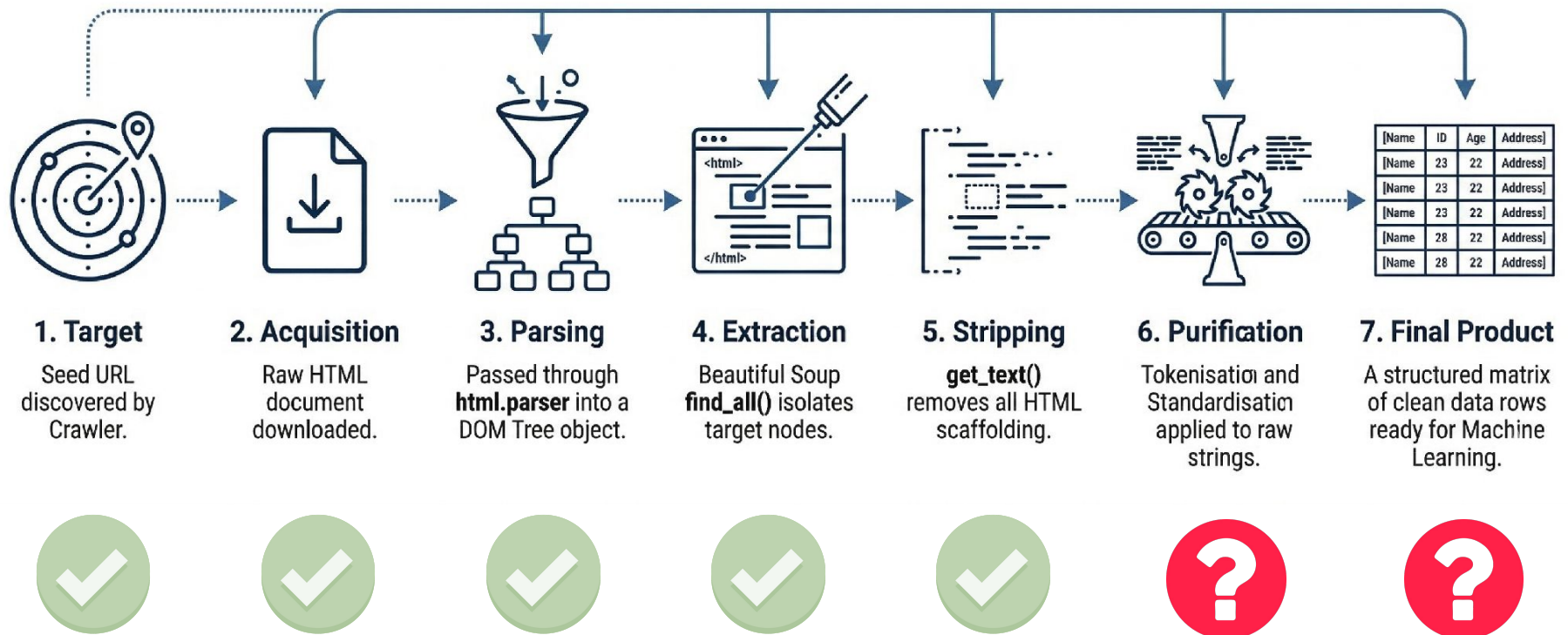
200gb data refresh monthly
on 27th then Guest pay
for own WIFI



The Complete Pipeline

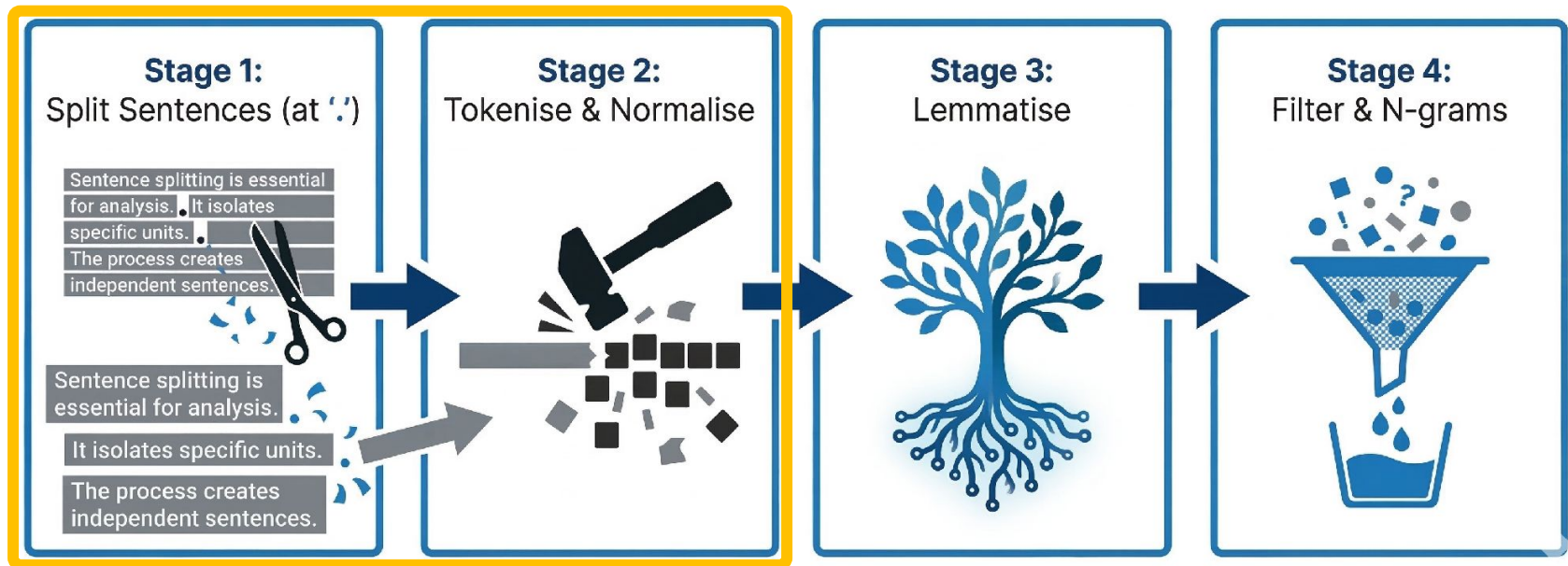


The Complete Pipeline



Text Preprocessing Sequence

(if we'd like to rely on words as features)



Sentence splitting

Where to split word strings into sentences?

- Split the string when two words are separated by one of “.”, “?” “!”, and (sometimes) the second string starts with a capital letter

John hit the ball. And then he ran.
Was John happy? Sure! Still, he looked sad.
She stared at him. *Was John happy?* she wondered.

Sentence splitting exceptions

When standard rules shatter...?

Condition	Input Example	Algorithm Action
Numbers/Currency mimic full stops.	It cost \$3.45 didn't it?	Do not split.
Mid-sentence abbreviations.	She mixed flour, etc. and poured.	Do not split.
Exceptions to Exceptions (Abbreviations ending a sentence).	...the dog, etc. February 1 is...	Force split.

Issues with names: J. K. Rowling, Kat I.P. Smith

Sentence splitting example

\$571 An extraordinary St Kilda mansion

Exterior walls feature crumbling pink paint, with an orange grove to greet you in the front garden. Beautifully and simply restored it has a French sensibility inside and an expansive garden. 4m high ceilings downstairs and upstairs. Rich with art and unique furnishings it has a glorious ambience inside

\$52 Melbourne, Brunswick, Aptmnt, CityViews

PrivateApartment. Medium to long stays. Bathroom & kitchen. Suits work from home. Brunswick is North of the city, but very close to the CBD. Light filled FULLY FURNISHED apartment in lovely building, includes linen. Tram and bus stops to hospitals and universities at front of building. Nearby: grocery, pharmacy, wine, nuts, bakery, restaurants, and take-aways. 200gb data refresh monthly on 27th then Guest pay for own WIFI

Sentence splitting example: NLTK

```
import nltk
from nltk.tokenize import sent_tokenize

nltk.download("punkt")
nltk.download("punkt_tab")

text = """PrivateApartment. {...}"""

sentences = sent_tokenize(text)

for sentence in sentences:
    print(sentence)
```

\$571 An extraordinary St Kilda mansion

Exterior walls feature crumbling pink paint, with an orange grove to greet you in the front garden. Beautifully and simply restored it has a French sensibility inside and an expansive garden. 4m high ceilings downstairs and upstairs. Rich with art and unique furnishings it has a glorious ambience inside

\$52 Melbourne, Brunswick, Aptmnt, CityViews

PrivateApartment. Medium to long stays. Bathroom & kitchen. Suits work from home. Brunswick is North of the city, but very close to the CBD. Light filled FULLY FURNISHED apartment in lovely building, includes linen. Tram and bus stops to hospitals and universities at front of building. Nearby: grocery, pharmacy, wine, nuts, bakery, restaurants, and take-aways. 200gb data refresh monthly on 27th then Guest pay for own WIFI

Sentence splitting example: Spacy

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = """PrivateApartment. {...}"""

doc = nlp(text)

sentences = [sent.text.strip() for sent in doc.sents]

for sentence in sentences:
    print(sentence)
```

\$571 An extradordinary St Kilda mansion

Exterior walls feature crumbling pink paint, with an orange grove to greet you in the front garden. Beautifully and simply restored it has a French sensibility inside and an expansive garden. 4m high ceilings downstairs and upstairs. Rich with art and unique furnishings it has a glorious ambience inside

\$52 Melbourne, Brunswick, Aptmnt, CityViews

PrivateApartment. Medium to long stays. Bathroom & kitchen. Suits work from home. Brunswick is North of the city, but very close to the CBD. Light filled FULLY FURNISHED apartment in lovely building, includes linen. Tram and bus stops to hospitals and universities at front of building. Nearby: grocery, pharmacy, wine, nuts, bakery, restaurants, and take-aways. 200gb data refresh monthly on 27th then Guest pay for own WIFI

Sentence splitting example: RegExp

```
import re

text = """PrivateApartment. {...}"""

sentences = re.split(r'(?<=[.!?])\s*(?=[A-Z0-9])', text)

for sentence in sentences:
    print(sentence)
```

More flexible but requires
your input &
understanding :-)

\$571 An extraordinary St Kilda mansion

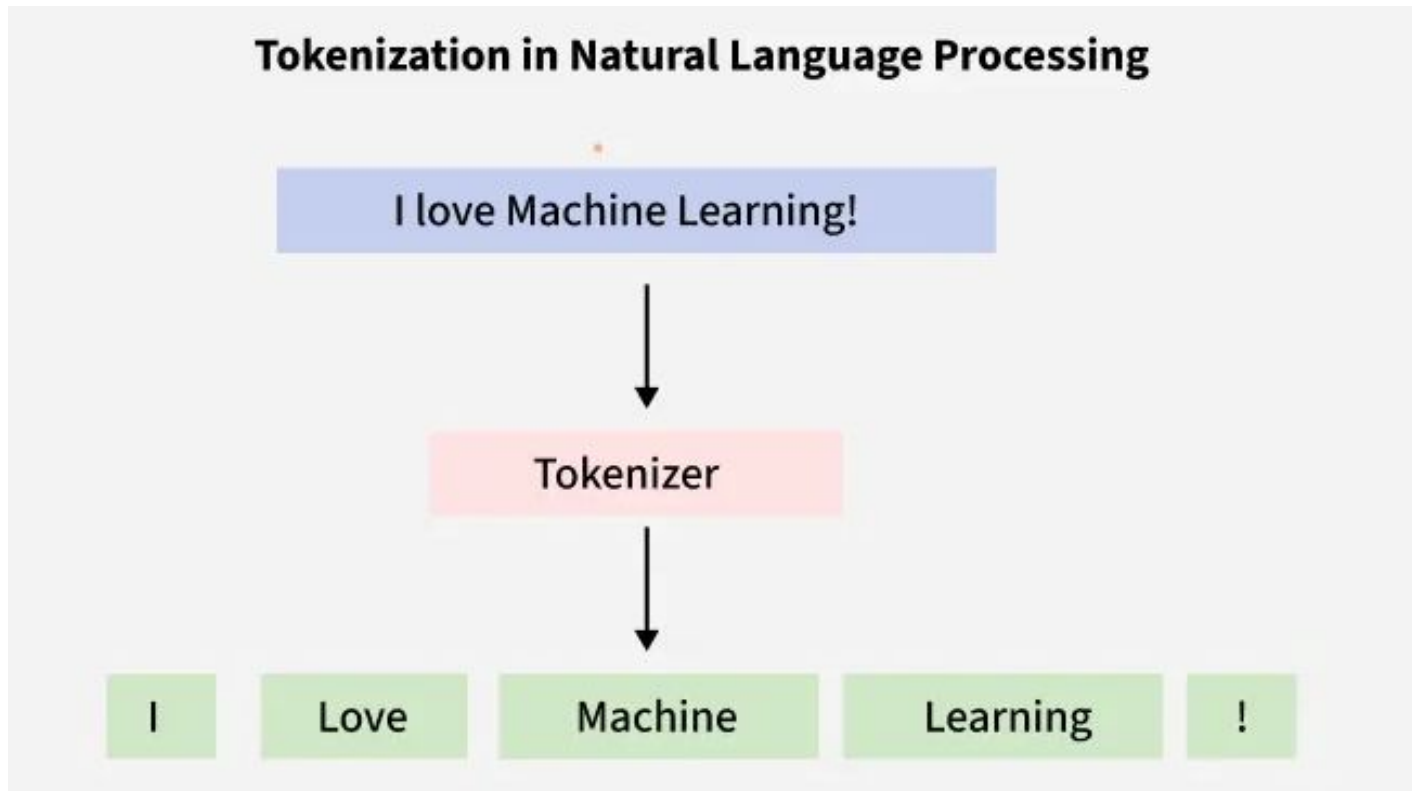
Exterior walls feature crumbling pink paint, with an orange grove to greet you in the front garden. Beautifully and simply restored it has a French sensibility inside and an expansive garden. 4m high ceilings downstairs and upstairs. Rich with art and unique furnishings it has a glorious ambience inside

\$52 Melbourne, Brunswick, Aptmnt, CityViews

PrivateApartment. Medium to long stays. Bathroom & kitchen. Suits work from home. Brunswick is North of the city, but very close to the CBD. Light filled FULLY FURNISHED apartment in lovely building, includes linen. Tram and bus stops to hospitals and universities at front of building. Nearby: grocery, pharmacy, wine, nuts, bakery, restaurants, and take-aways. 200gb data refresh monthly on 27th then Guest pay for own WIFI

Word Tokenisation / segmentation

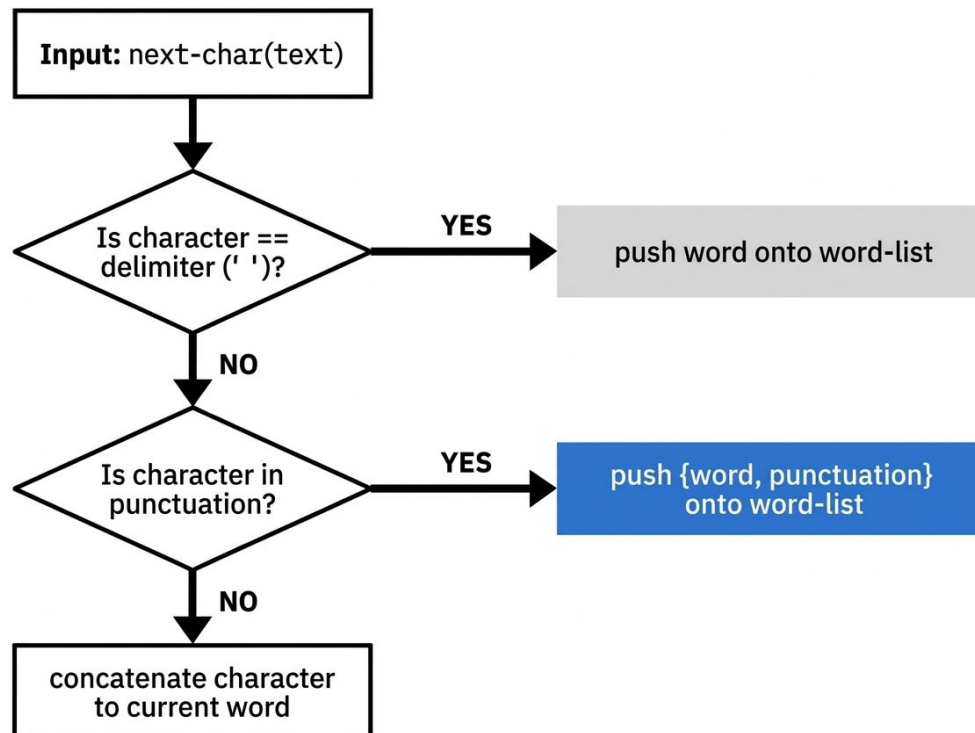
Split a (sentence) string into word-like tokens



Word Tokenisation / segmentation

Split a (sentence) string into word-like tokens

- **Delimiter** := “ ”
- Strip off **punctuation** as separate words
 - Punctuation := “.” “,” “;” “:” “?” “!” “-” “<” “>” “/” “|” “+” “\$” “%” “~”



Sentence splitting example: NLTK

```
import nltk
from nltk.tokenize import sent_tokenize

nltk.download("punkt")
nltk.download("punkt_tab")

text = """PrivateApartment. {...}"""

sentences = sent_tokenize(text)

for sentence in sentences:
    print(sentence)
```

\$52 Melbourne, Brunswick, Aptmnt, CityViews

PrivateApartment. Medium to long stays. Bathroom & kitchen. Suits work from
home. Brunswick is North of the city, but very close to the CBD. Light filled FULLY FURNISHED

...

Sentence splitting example: Spacy

```
import spacy

# Load the English model
nlp = spacy.load("en_core_web_sm")

text = "PrivateApartment. {...}"

doc = nlp(text)

tokens = [token.text for token in doc]

print(tokens)
```

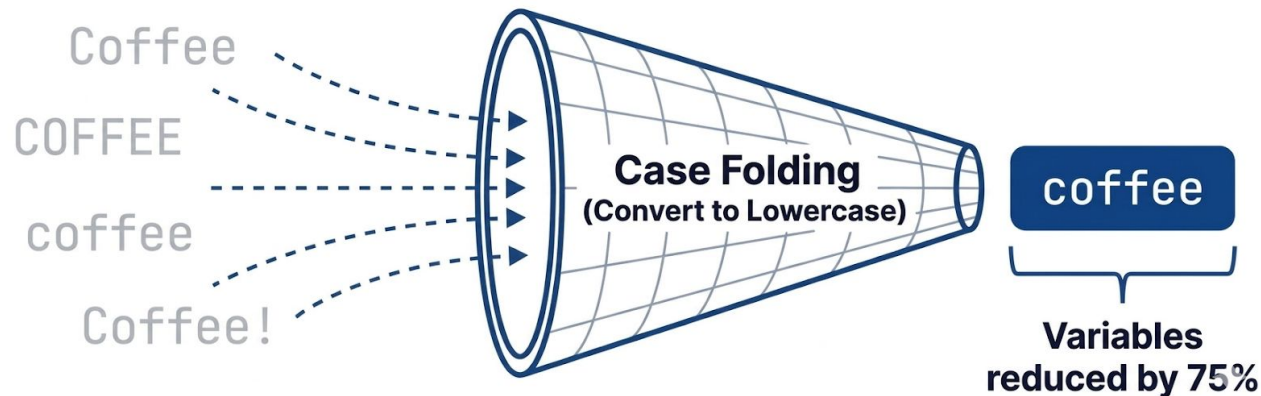
\$52 Melbourne, Brunswick, Aptmnt, CityViews

PrivateApartment. Medium to long stays. Bathroom & kitchen. Suits work from
home. Brunswick is North of the city, but very close to the CBD. Light filled FULLY FURNISHED

...

Case folding (and the sparsity problem)

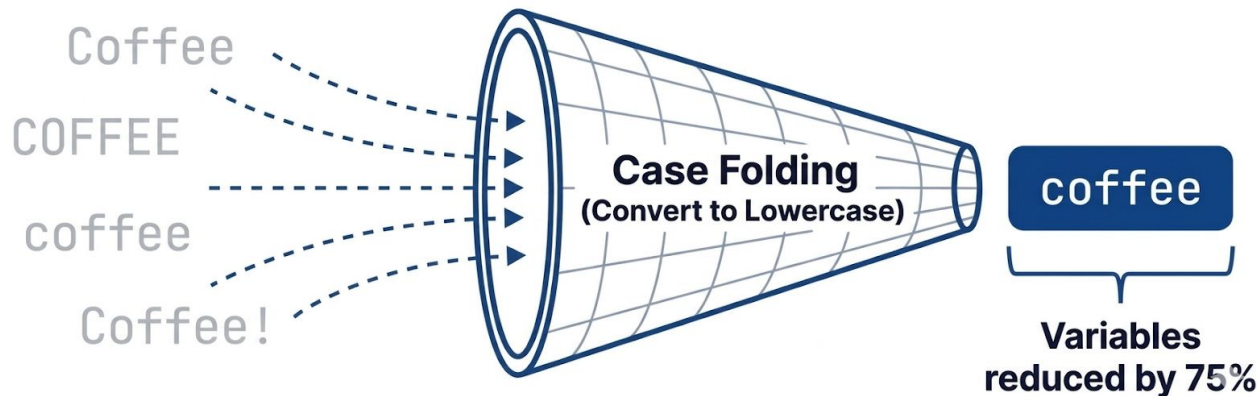
Convert text to single case (no uppercase)



- By mapping all variations to a single lower-case form, we drastically **reduce data sparsity**.
- This is a simple, highly effective technique to support search, matching, and pattern recognition.

Case folding (and the sparsity problem)

Convert text to single case (no uppercase)



Apple
VS
apple



- By mapping all variations to a single lower-case form, we drastically **reduce data sparsity**.
- This is a simple, highly effective technique to support search, matching, and pattern recognition.

Case folding (and the sparsity problem)

Convert text to single case (no uppercase; remove punctuation)

```
doc = nlp(text)

tokens = [
    token.text.lower()
    for token in doc
    if not token.is_punct and not token.is_space
]
```

['privateapartment', 'medium', 'to', 'long', 'stays', 'bathroom', 'kitchen', 'suits', 'work', 'from', 'home', 'brunswick', 'is', 'north', 'of', 'the', 'city', 'but', 'very', 'close', 'to', 'the', 'cbd', 'light', 'filled', 'fully', 'furnished', 'apartment', 'in', 'lovely', 'building', 'includes', 'linen', 'tram', 'and', 'bus', 'stops', 'to', 'hospitals', 'and', 'universities', 'at', 'front', 'of', 'building', 'nearby', 'grocery', 'pharmacy', 'wine', 'nuts', 'bakery', 'restaurants', 'and', 'take', 'aways', '200', 'gb', 'data', 'refresh', 'monthly', 'on', '27th', 'then', 'guest', 'pay', 'for', 'own', 'wifi']

- By mapping all variations to a single lower-case form, we drastically **reduce data sparsity**.
- This is a simple, highly effective technique to support search, matching, and pattern recognition.

Case folding (and the sparsity problem)

Convert text to single case (no uppercase; remove punctuation)

```
doc = nlp(text)

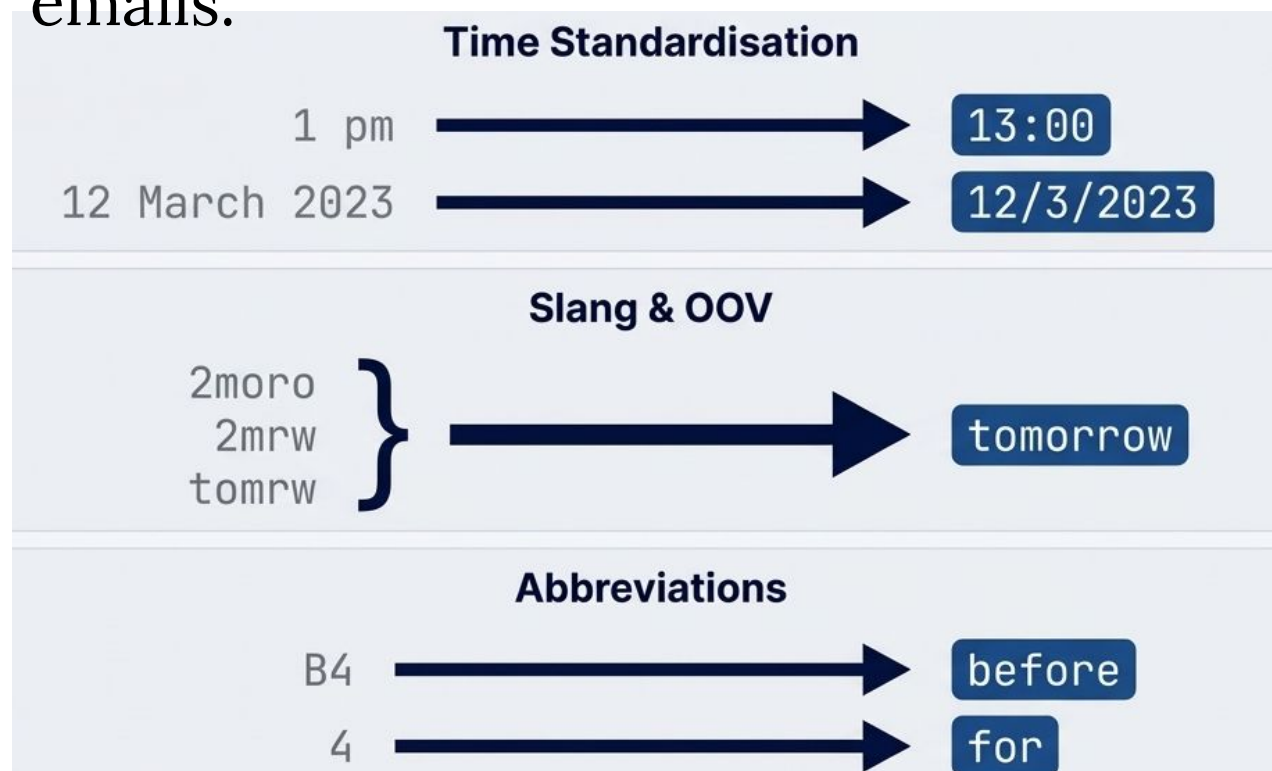
tokens = [
    token.text.lower()
    for token in doc
    if not token.is_punct and not token.is_space
]
```

['privateapartment', 'medium', 'to', 'long', 'stays', 'bathroom', 'kitchen', 'suits', 'work', 'from', 'home', 'brunswick', 'is', 'north', 'of', 'the', 'city', 'but', 'very', 'close', 'to', 'the', 'cbd', 'light', 'filled', 'fully', 'furnished', 'apartment', 'in', 'lovely', 'building', 'includes', 'linen', 'tram', 'and', 'bus', 'stops', 'to', 'hospitals', 'and', 'universities', 'at', 'front', 'of', 'building', 'nearby', 'grocery', 'pharmacy', 'wine', 'nuts', 'bakery', 'restaurants', 'and', 'take', 'always', '200', 'gb', 'data', 'refresh', 'monthly', 'on', '27th', 'then', 'guest', 'pay', 'for', 'own', 'wifi']

- By mapping all variations to a single lower-case form, we drastically **reduce data sparsity**.
- This is a simple, highly effective technique to support search, matching, and pattern recognition.

Other Text Normalisation

Crucial for processing noisy, real-world data like social media comments, text messages and emails.



***OOV = Out-of-Vocab**

Case folding (and the sparsity problem)

Convert text to single case (no uppercase; remove punctuation)

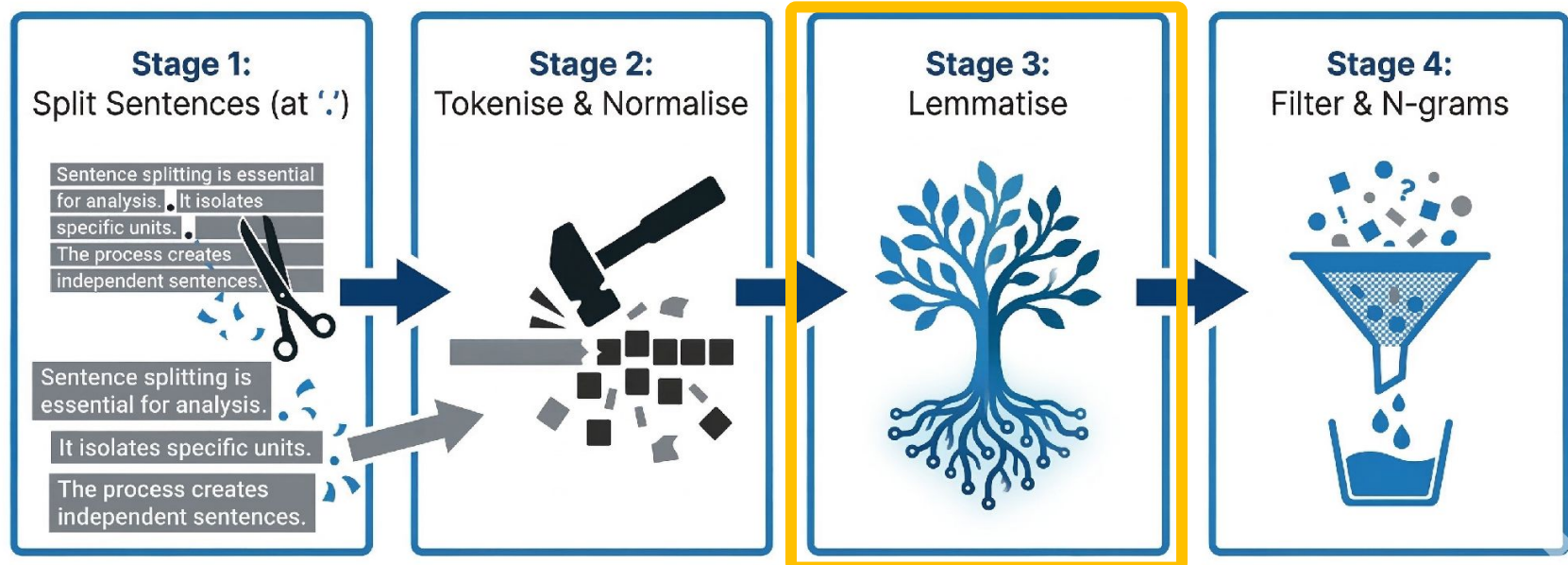
```
doc = nlp(text)

tokens = [
    token.text.lower()
    for token in doc
    if not token.is_punct and not token.is_space
]
```

['private', 'apartment', 'medium', 'to', 'long', 'stays', 'bathroom', 'kitchen', 'suits', 'work', 'from', 'home', 'brunswick', 'is', 'north', 'of', 'the', 'city', 'but', 'very', 'close', 'to', 'the', 'cbd', 'light', 'filled', 'fully', 'furnished', 'apartment', 'in', 'lovely', 'building', 'includes', 'linen', 'tram', 'and', 'bus', 'stops', 'to', 'hospitals', 'and', 'universities', 'at', 'front', 'of', 'building', 'nearby', 'grocery', 'pharmacy', 'wine', 'nuts', 'bakery', 'restaurants', 'and', 'take', 'aways', '200', 'gigabytes', 'data', 'refresh', 'monthly', 'on', '27', 'then', 'guest', 'pay', 'for', 'own', 'wifi']

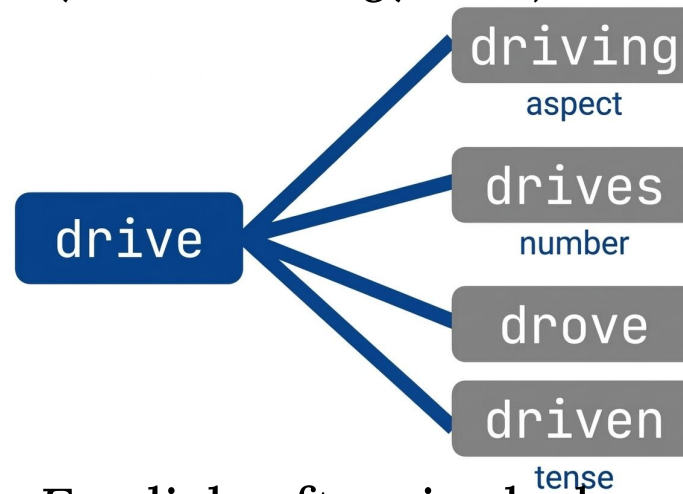
- By mapping all variations to a single lower-case form, we drastically **reduce data sparsity**.
- This is a simple, highly effective technique to support search, matching, and pattern recognition.

Text Preprocessing Sequence



Word Morphology

Problem 1: English has different forms of the same word, for **number** (singular/plural), **tense** (past/present/future/etc.), and **aspect** (completed/continuing/etc.)



Problem 2: Words in English often include a **root** or **stem** plus more parts (**morphemes**)

in + **expense** + ive = **inexpensive**

Treating each one separately makes your lexicon MUCH larger!

Word Morphology

LATIN — 1ST CONJUGATION

amō, amāre, amāvī, amātus

bencrowder.net • Last modified 17 Jul 2018

ACTIVE INDICATIVE

	PRESENT	IMPERFECT	FUTURE	PERFECT	PLUPERFECT	FUTURE PERFECT
1ST SG.	amō	amābam	amābō	amāvī	amāveram	amāverō
2ND SG.	amās	amābās	amābis	amāvisti	amāverās	amāveris
3RD SG.	amat	amābat	amābit	amāvit	amāverat	amāverit
1ST PL.	amāmus	amābāmus	amābimus	amāvimus	amāverāmus	amāverimus
2ND PL.	amātis	amābātis	amābitis	amāvistis	amāverātis	amāveritis
3RD PL.	amant	amābant	amābunt	amāverunt	amāverant	amāverint

PASSIVE INDICATIVE

	PRESENT	IMPERFECT	FUTURE	PERFECT	PLUPERFECT	FUTURE PERFECT
1ST SG.	amor	amābar	amābor	amātus sum	amātus eram	amātus erō
2ND SG.	amāris	amābāris	amāberis	amātus es	amātus erās	amātus eris
3RD SG.	amātur	amābātur	amābitur	amātus est	amātus erat	amātus erit
1ST PL.	amāmur	amābāmur	amābimur	amāti sumus	amāti erāmus	amāti erimus
2ND PL.	amāmini	amābāmini	amābimini	amāti estis	amāti erātis	amāti eritis
3RD PL.	amantur	amābantur	amābuntur	amāti sunt	amāti erant	amāti erunt

ACTIVE SUBJUNCTIVE

	PRESENT	IMPERFECT	PERFECT	PLUPERFECT
1ST SG.	amem	amārem	amāverim	amāvissem
2ND SG.	amēs	amārēs	amāveris	amāvisseēs
3RD SG.	amet	amāret	amāverit	amāvisset
1ST PL.	amēmus	amāremus	amāverimus	amāvissemus
2ND PL.	amētis	amāretis	amāveritis	amāvissetis
3RD PL.	ament	amārent	amāverint	amāvissent

PASSIVE SUBJUNCTIVE

	PRESENT	IMPERFECT	PERFECT	PLUPERFECT
1ST SG.	amer	amārer	amātus sim	amātus essem
2ND SG.	amēris	amārēris	amātus sis	amātus esseēs
3RD SG.	amētur	amārētur	amātus sit	amātus esset
1ST PL.	amēmur	amārēmur	amāti simus	amāti essemus
2ND PL.	amēmini	amārēmini	amāti sitis	amāti essētis
3RD PL.	amentur	amārentur	amāti sint	amāti essent

INFINITIVE

	PRESENT	FUTURE	PERFECT
ACTIVE	amāre	amātūrus esse	amāvisse
PASSIVE	amārī	amātum iri	amātus esse

PARTICIPLE

	PRESENT	FUTURE	PERFECT
ACTIVE	amāns	amātūrus	
PASSIVE		amandus	amātus

GERUND

GEN.	amandī
DAT.	amandō
ACC.	amandum
ABL.	amandō

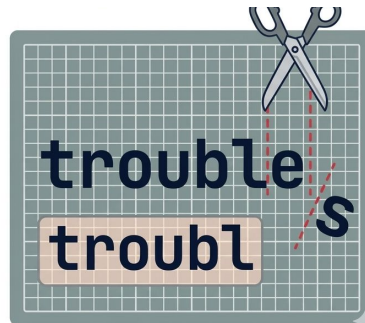
IMPERATIVE

	SINGULAR	PLURAL
ACTIVE	amā	amāte
PASSIVE	amāre	amāmini

Stemming: the Brute Force

- Stemming replaces word suffixes to (try to) produce a common **root form**
- Results **do not necessarily** look like proper words
- **Porter stemmer**, one of the most widely used tools;

Porter Stemmer Rules		
sses	→	ss
ies	→	i
tional	→	tion



- language-specific rules
- overgeneralises: wing → w (removes -ing)
- miss irregulars: go-went-gone

Porter stemmer: <https://tartarus.org/martin/PorterStemmer/>

NLTK toolkit stemmer: <https://www.nltk.org/howto/stem.html>

Stemming: NLTK

```
from nltk.stem import PorterStemmer

words = [
    'private', 'apartment', 'medium', ..
]

stemmer = PorterStemmer()

stemmed_words = [stemmer.stem(word) for word in words]

print(stemmed_words)
```

```
['privat', 'apart', 'medium', 'to', 'long', 'stay', 'bathroom', 'kitchen', 'suit', 'work', 'from', 'home',
'brunswick', 'is', 'north', 'of', 'the', 'citi', 'but', 'veri', 'close', 'to', 'the', 'cbd', 'light', 'fill', 'fulli',
'furnish', 'apart', 'in', 'love', 'build', 'includ', 'linen', 'tram', 'and', 'bu', 'stop', 'to', 'hospit', 'and',
'univers', 'at', 'front', 'of', 'build', 'nearbi', 'groceri', 'pharmaci', 'wine', 'nut', 'bakeri', 'restaur',
'and', 'take', 'away', '200', 'gigabyt', 'data', 'refresh', 'monthli', 'on', '27', 'then', 'guest', 'pay', 'for',
'own', 'wifi']
```

Porter stemmer: <https://tartarus.org/martin/PorterStemmer/>

NLTK toolkit stemmer: <https://www.nltk.org/howto/stem.html>

Stemming: NLTK

Cannot map into real dictionaries/lexicons

```
from nltk.stem import PorterStemmer

words = [
    'private', 'apartment', 'medium', ..
]

stemmer = PorterStemmer()

stemmed_words = [stemmer.stem(word) for word in words]

print(stemmed_words)
```

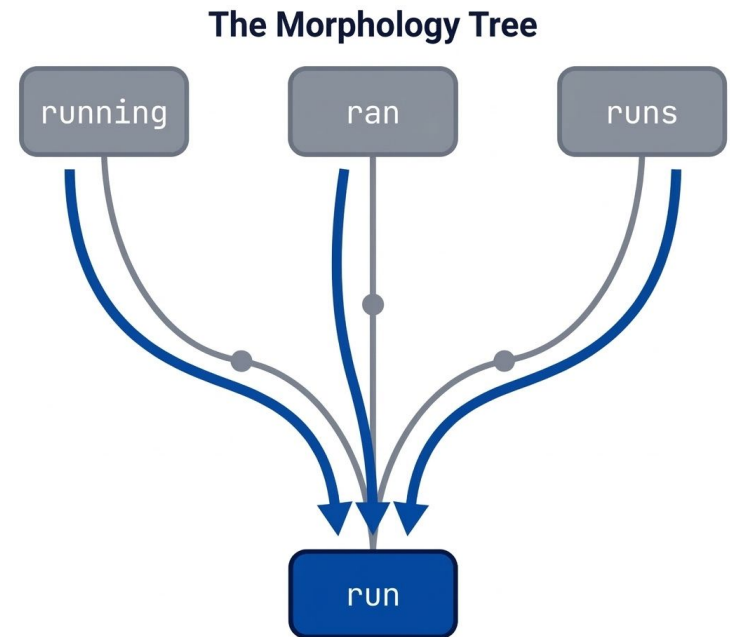
['privat', 'apart', 'medium', 'to', 'long', 'stay', 'bathroom', 'kitchen', 'suit', 'work', 'from', 'home', 'brunswick', 'is', 'north', 'of', 'the', 'citi', 'but', 'veri', 'close', 'to', 'the', 'cbd', 'light', 'fill', 'fulli', 'furnish', 'apart', 'in', 'love', 'build', 'includ', 'linen', 'tram', 'and', 'bu', 'stop', 'to', 'hospit', 'and', 'univers', 'at', 'front', 'of', 'build', 'nearbi', 'groceri', 'pharmaci', 'wine', 'nut', 'bakeri', 'restaur', 'and', 'take', 'away', '200', 'gigabyt', 'data', 'refresh', 'monthli', 'on', '27', 'then', 'guest', 'pay', 'for', 'own', 'wifi']

Porter stemmer: <https://tartarus.org/martin/PorterStemmer/>

NLTK toolkit stemmer: <https://www.nltk.org/howto/stem.html>

Lemmatising

- Remove inflections and map a word to its *proper root* (dictionary) form AKA lemma
- requires extensive linguistic input: dictionaries, grammars, etc



Approaches to Lemmatising

Manually built morphological analysers: use tons of carefully **built rules**

- Really necessary for German and other heavily inflected languages
- Finite-state Transducers: [HFST](#), [Apertium](#), [FOMA](#)

Machine learned lemmatisers: algorithm trained to learn when and how to convert words

- ❤️ UniMorph: <https://unimorph.github.io/> (a multilingual dataset of inflectional paradigms)
- ❤️ [SIGMORPHON Shared Task 2020](#) (Vylomova et al., 2020)

Python **NLTK** provides **WordNet Lemmatiser** that uses the WordNet database to look up lemmas of words

- <https://www.guru99.com/stemming-lemmatization-python-nltk.html>

Lemmatisation: Spacy

```
import spacy

nlp = spacy.load("en_core_web_sm")

words = ['private', 'apartment', ...]

doc = nlp(" ".join(words))

lemmas = [token.lemma_.lower() for token in doc]

print(lemmas)
```

```
['private', 'apartment', 'medium', 'to', 'long', 'stay', 'bathroom', 'kitchen', 'suit', 'work', 'from',  
'home', 'brunswick', 'be', 'north', 'of', 'the', 'city', 'but', 'very', 'close', 'to', 'the', 'cbd', 'light', 'fill',  
'fully', 'furnish', 'apartment', 'in', 'lovely', 'building', 'include', 'linen', 'tram', 'and', 'bus', 'stop',  
'to', 'hospital', 'and', 'university', 'at', 'front', 'of', 'build', 'nearby', 'grocery', 'pharmacy', 'wine',  
'nut', 'bakery', 'restaurant', 'and', 'take', 'away', '200', 'gigabyte', 'datum', 'refresh', 'monthly',  
'on', '27', 'then', 'guest', 'pay', 'for', 'own', 'wifi']
```

Lemmatisation: Spacy

```
import spacy

nlp = spacy.load("en_core_web_sm")

words = ['private', 'apartment', ...]

doc = nlp(" ".join(words))

lemmas = [token.lemma_.lower() for token in doc]

print(lemmas)
```

```
['private', 'apartment', 'medium', 'to', 'long', 'stay', 'bathroom', 'kitchen', 'suit', 'work', 'from',  
'home', 'brunswick', 'be', 'north', 'of', 'the', 'city', 'but', 'very', 'close', 'to', 'the', 'cbd', 'light', 'fill',  
'fully', 'furnish', 'apartment', 'in', 'lovely', 'building', 'include', 'linen', 'tram', 'and', 'bus', 'stop',  
'to', 'hospital', 'and', 'universit',  
'nut', 'bakery', 'restaurant', 'ar',  
'on', '27', 'then', 'guest', 'pay', 'f']
```

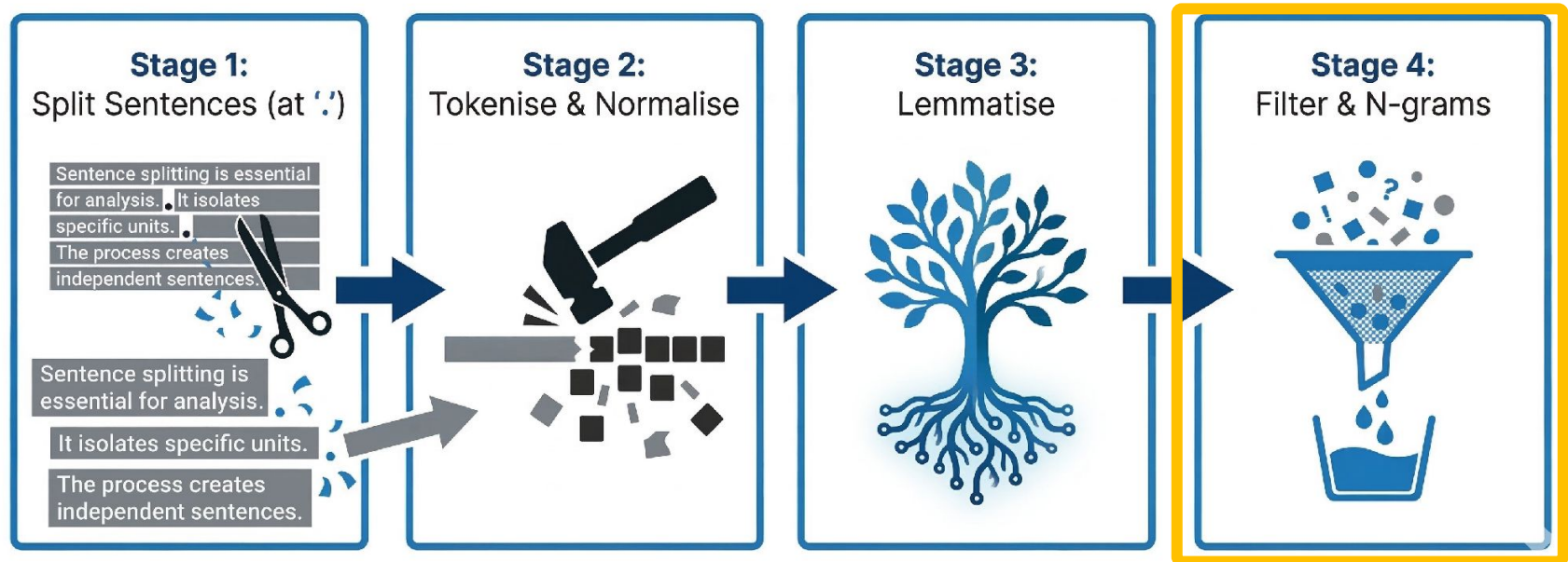
Lemmas are useful for
lexicon-based feature
extraction (e.g., positive and
negative sentiment)

Stemming vs lemmatising

Choosing a regularisation strategy

	Stemming	Lemmatisation
Mechanism	Algorithmic suffix chopping	Dictionary-based analysis
Speed & Complexity	Fast execution Extremely simple code	Slower execution Requires extensive language databases
Output Validity	Often yields non-words or fragments	Always yields valid dictionary lemmas
Tool Example	Porter Stemmer	NLTK WordNet
Input → Output Example	eating → eat (success) wing → w (failure)	eating → eat (success) green → green (success)

Text Preprocessing Sequence



Open- and closed-class words

Keep: Open-Class Words

Content Words

(nouns, verbs, adjectives, adverbs)

bicycle

tree

eat

drink

happy

quickly

Carries the actual meaning. Infinite in number.

Discard: Closed-Class Words

Function Words / Stop Words

(determiners, pronouns, prepositions, conjunctions...)

the

to

not

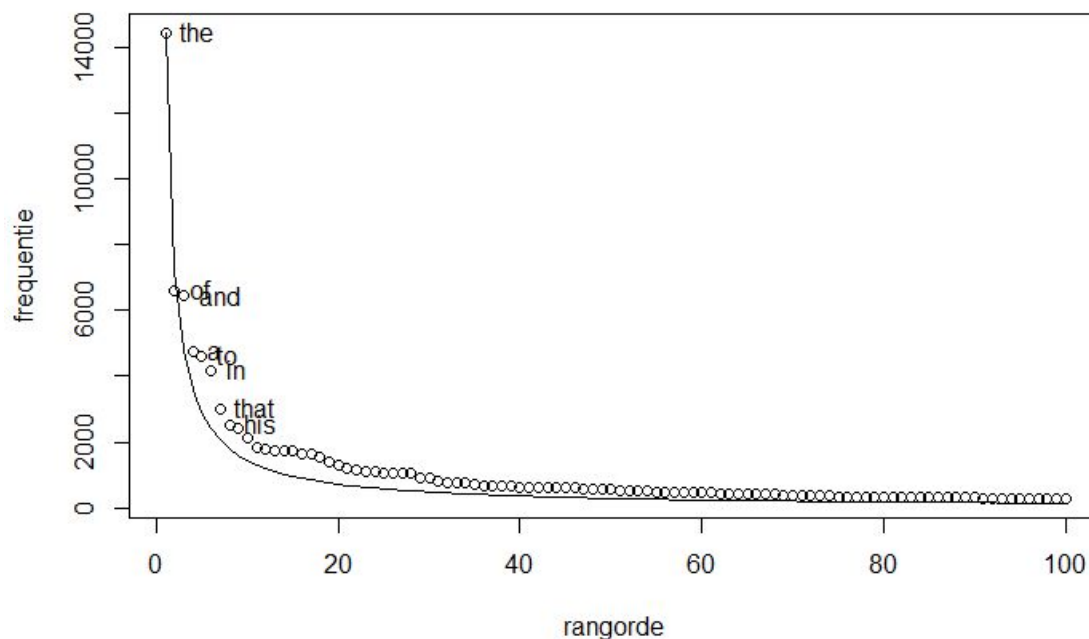
and

Grammatical Scaffolding. Only a few dozen exist.
Usually removed to improve topic clustering and counting.

Logarithmic transformation

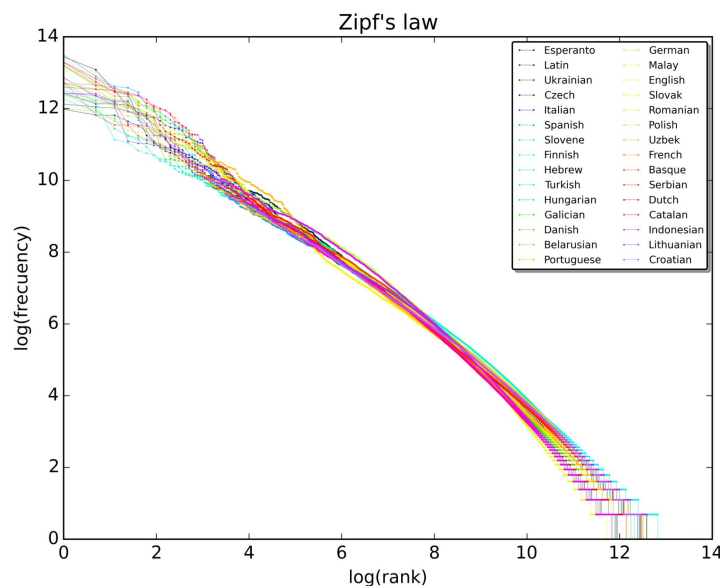
- **log-likelihood** (products→sums; easier to compute). Often used in LMs optimization (MLE)
- **Zipf's Law** (word frequency distributions)

1. the
2. and
3. of
4. a
5. to
6. that
7. this
8. ...



Logarithmic transformation

- **log-likelihood** (products → sums; easier to compute). Often used in LMs optimization (MLE)
- **Zipf's Law** (word frequency distributions)



$$\text{word frequency} \propto \frac{1}{\text{word rank}}$$

Zipf's law plot for the first 10 million words in 30 Wikipedias

Stopword lists

Stopwords are the closed-class (function) words you want to remove

Why? Reduce the number of features/words, support more accurate clustering

When? Commonly applied before searching, text classification, topic modelling, topic extraction, etc.

Standard Stopword Lists

There are many lists of stopwords

- **ranks.nl** (40 languages): <https://www.ranks.nl/stopwords>

RANKS NL >> Request Free Plan Demo Plans & Pricing More ▾

Stopword Lists

Collection of stopwords lists in **40+** languages. Find the English stopwords below and/or follow the links to view our other language stop word lists.

English Stopwords

Default English stopwords list

This list is used in our [Page Analyzer](#) and [Article Analyzer](#) for English text, when you let it use the default stopwords list.

a	ourselves
about	out
above	over
after	own
again	same
against	shan't
all	she
am	she'd
an	she'll
and	she's
any	should
are	shouldn't

Stopwords in other languages

- [Arabic](#)
- [Armenian](#)
- [Basque](#)
- [Bengali](#)
- [Brazilian](#)
- [Bulgarian](#)
- [Catalan](#)
- [Chinese](#)
- [Croatian](#)
- [Czech](#)
- [Danish](#)
- [Dutch](#)
- [Finnish](#)

Stopword Examples

{‘ourselves’, ‘hers’, ‘between’, ‘yourself’, ‘but’, ‘again’, ‘there’, ‘about’, ‘once’, ‘during’, ‘out’, ‘very’, ‘having’, ‘with’, ‘they’, ‘own’, ‘an’, ‘be’, ‘some’, ‘for’, ‘do’, ‘its’, ‘yours’, ‘such’, ‘into’, ‘of’, ‘most’, ‘itself’, ‘other’, ‘off’, ‘is’, ‘s’, ‘am’, ‘or’, ‘who’, ‘as’, ‘from’, ‘him’, ‘each’, ‘the’, ‘themselves’, ‘until’, ‘below’, ‘are’, ‘we’, ‘these’, ‘your’, ‘his’, ‘through’, ‘don’, ‘nor’, ‘me’, ‘were’, ‘her’, ‘more’, ‘himself’, ‘this’, ‘down’, ‘should’, ‘our’, ‘their’, ‘while’, ‘above’, ‘both’, ‘up’, ‘to’, ‘ours’, ‘had’, ‘she’, ‘all’, ‘no’, ‘when’, ‘at’, ‘any’, ‘before’, ‘them’, ‘same’, ‘and’, ‘been’, ‘have’, ‘in’, ‘will’, ‘on’, ‘does’, ‘yourselves’, ‘then’, ‘that’, ‘because’, ‘what’, ‘over’, ‘why’, ‘so’, ‘can’, ‘did’, ‘not’, ‘now’, ‘under’, ‘he’, ‘you’, ‘herself’, ‘has’, ‘just’, ‘where’, ‘too’, ‘only’, ‘myself’, ‘which’, ‘those’, ‘i’, ‘after’, ‘few’, ‘whom’, ‘t’, ‘being’, ‘if’, ‘theirs’, ‘my’, ‘against’, ‘a’, ‘by’, ‘doing’, ‘it’, ‘how’, ‘further’, ‘was’, ‘here’, ‘than’}

A stopwords list can be custom-made for a specific context/domain.

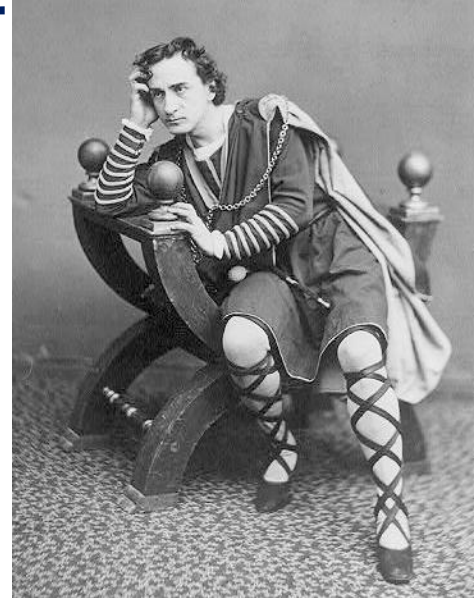
Remove Stopwords: Spacy

```
import spacy
from spacy.lang.en.stop_words import
STOP_WORDS

words = [ 'private', 'apartment', 'medium', ]

filtered_words = [
    word for word in words
    if word not in STOP_WORDS
]

print(filtered_words)
```



“To be or not to be?”

['private', 'apartment', 'medium', 'to', 'long', 'stay', 'bathroom', 'kitchen', 'suit', 'work', 'from', 'home', 'brunswick', 'be', 'north', 'of', 'the', 'city', 'but', 'very', 'close', 'to', 'the', 'cbd', 'light', 'fill', 'fully', 'furnish', 'apartment', 'in', 'lovely', 'building', 'include', 'linen', 'tram', 'and', 'bus', 'stop', 'to', 'hospital', 'and', 'university', 'at', 'front', 'of', 'build', 'nearby', 'grocery', 'pharmacy', 'wine', 'nut', 'bakery', 'restaurant', 'and', 'take', 'away', '200', 'gigabyte', 'datum', 'refresh', 'monthly', 'on', '27', 'then', 'guest', 'pay', 'for', 'own', 'wifi']

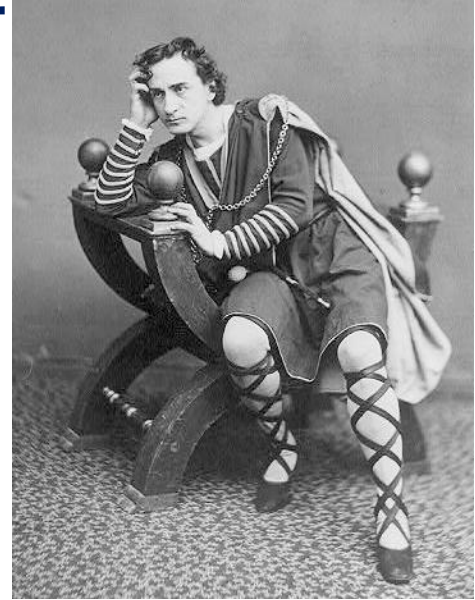
Remove Stopwords: Spacy

```
import spacy
from spacy.lang.en.stop_words import
STOP_WORDS

words = [ 'private', 'apartment', 'medium', ]

filtered_words = [
    word for word in words
    if word not in STOP_WORDS
]

print(filtered_words)
```



“To be or not to be?”

['private', 'apartment', 'medium', 'to', 'long', 'stay', 'bathroom', 'kitchen', 'suit', 'work', 'from', 'home', 'brunswick', 'be', 'north', 'of', 'the', 'city', 'but', 'very', 'close', 'to', 'the', 'cbd', 'light', 'fill', 'fully', 'furnish', 'apartment', 'in', 'lovely', 'building', 'include', 'linen', 'tram', 'and', 'bus', 'stop', 'to', 'hospital', 'and', 'university', 'at', 'front', 'of', 'build', 'nearby', 'grocery', 'pharmacy', 'wine', 'nut', 'bakery', 'restaurant', 'and', 'take', 'away', '200', 'gigabyte', 'datum', 'refresh', 'monthly', 'on', '27', 'then', 'guest', 'pay', 'for', 'own', 'wifi']

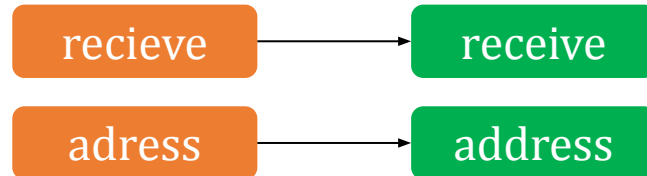
Exercise

Eat. Think. Move at Melbourne Connect is **designed to promote wellness and a healthy lifestyle in our community**. This program features fitness classes, cooking workshops, information sessions, etc. and is designed for you! Each class costs only \$5.99 and won't break your budget but will leave you feeling great.

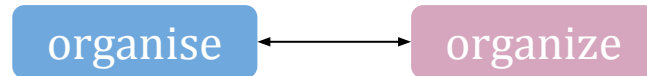
1. Write code to split the text into sentences
2. Then split each sentence into words
3. Then replace words with their dict forms (lemmas)
4. Remove all the stopwords

Text is still very messy!

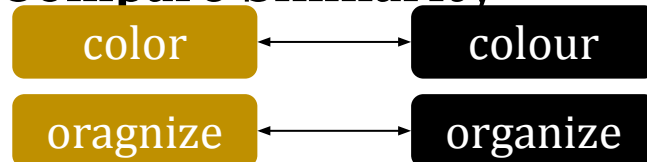
- **Fix Spelling errors**



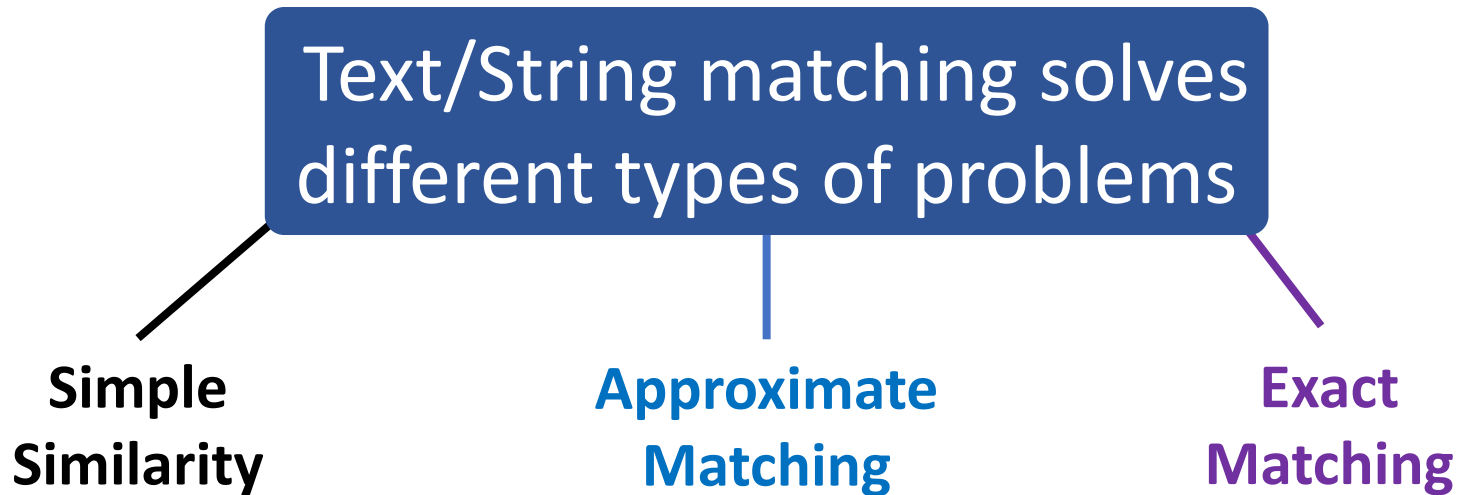
- **Language Variation and slang (e.g, American vs British English)**



- **Compare Similarity**



Text is still very messy!



Simple Similarity: n-grams

When? Quick, Simple Similarity Checking

- **(Word) ngram:** A series of length n words

Two-word ngrams: “Humpty Dumpty” “coffee break”

- **Character ngram:** A substring of length n letters

Let G_n be a function that returns all character ngrams (of length n)

Some letter ngrams:

- G_2 : bi-grams of **color**: [$\#c$, co , ol , lo , or , $r\#$]
- G_3 : tri-grams of **color**: [$\#\#c$, $\#co$, col , olo , lor , $or\#$, $r\#\#$]

*‘#’ is a padding character

How similar are two strings in bi-grams?

Let G_n be a function that returns all char-ngrams (of length n).

Character ngram distance between x and y :

$$|G_n(x)| + |G_n(y)| - 2 \times |G_n(x) \cap G_n(y)|$$

$G_2(\text{google}) = [\#g, go, oo, og, gl, le, e\#]$

$G_2(\text{goggle}) = [\#g, go, og, gg, gl, le, e\#]$

$|G_2(\text{google})| = 7,$

$|G_2(\text{goggle})| = 7,$

$|G_2(\text{google}) \cap G_2(\text{goggle})| = 5$

Bi-gram distance between google and goggle

$$= |G_2(\text{google})| + |G_2(\text{goggle})| - 2 \times |G_2(\text{google}) \cap G_2(\text{goggle})|$$

$$= 7 + 7 - 2 \times 5$$

$$= 4$$

identical  dissimilar
0 larger

Approximate Matching: Edit distance

When? When text is not exactly the same

Transform the source string into the target string by applying **three basic character operations**:

Insert, Delete, Replace

- Each operation incurs a cost (a mismatch score)
 - **The more edits you need, the further the distance**

- Ex:

“... if you can see no ther option ...”

other	
here	
otter	

Approximate Matching: Edit distance

When? When text is not exactly the same

Transform the source string into the target string by applying **three basic character operations**:

Insert, Delete, Replace

- Each operation incurs a cost (a mismatch score)
 - The more edits you need, the larger the distance

• Ex:

“... if you can see no ther option ...”



other	1 insert
here	1 insert
	1 delete
otter	1 insert, 1 replace



Similarity = 1 – edit distance

- *Normalised distance* = the distance divided by the length of the longer string

$$\frac{d(S1, S2)}{\max(|S1|, |S2|)}$$

- *Similarity = 1 – normalised distance*

$$simedit(s1, s2) = 1.0 - \frac{d(S1, S2)}{\max(|S1|, |S2|)}$$

- ther → otter?

	t	h	e	r
o	t	t	e	r

$$simedit(ther, otter) = 1.0 - \frac{2}{5} = 0.6$$

Many similarity measures

Many metrics to measure similarity or distance, incl.:

- Jaccard
- Sørensen-Dice
- Cosine
- Jaro-Winker
- Edit distance metrics
 - Levenshtein
 - Longest common substring
 - Hamming

Similarity = 1 – distance

Regular Expressions:

Finding specific things in text

Assume you have a large collection of unstructured text data and need to write wrangling code to:

- Check whether it contains any IP addresses (e.g. 128.250.65.5)
- Find all the telephone numbers (e.g. 0414 555 555)
- Find all the company names (e.g. “Gear Cutters Pty. Ltd.”)
- Requirements
 - Do it succinctly
 - Do it unambiguously
 - Have maintainable code

Regular Expressions:

Patterns in text

Regular expression: a pattern that can match all kinds of substrings, depending on the rules you specify

- *Look for capital and/or lowercase letters*
- *Look for numbers or letters or other symbols*
- *Match or skip any number of symbols (= “wildcards”)*
- *...and many more*

Uses:

- Find desired items
- Calculate statistics (count occurrences of items)
- Check for integrity, filtering, substitutions ...

RE: Basic matching

- Characters match themselves exactly
 - The pattern **t** will match the string “t”
- Strings match themselves exactly
 - The pattern **hello** will match the string “hello”
 - **Hello** will match the string “Hello”
- **.** : the **wildcard** matches any character
 - For example, **a.c** matches “a/c”, “abc”, “a5c”
 - So **.** is a ‘special’ character (= metacharacter)

RE: Matching character sets

- `[]`: matches any character from a set of characters
 - `[abc]`, `[a-zA-Z]` matches any string using **just** those letters
 - `+` means one or more occurrences

```
>>> import re
>>> text = 'Hi, my name is Mel! What is yours?'
>>> name = re.findall(r'name is ([A-Z][a-z]+)', text)
>>> name
['Mel']
```

- `[^]`:
 - adding `^` as the **first** character in the set matches the set complement (all the others **except** it)
 - `[^z]` matches anything **but** “z”

RE: Predefined sets

- Some predefined named special character sets:
e.g.,
 - **\d** any decimal digit == [0-9]
 - **\w** any alphanumeric character == [a-zA-Z0-9_]
 - **\W** any non-alphanumeric character ==
[[^]a-zA-Z0-9_]

```
In [1]: import re

In [2]: text = 'This is COMP20008'

In [3]: digit = re.findall(r'\d', text)

In [4]: digit
Out[4]: ['2', '0', '0', '0', '8']
```

- Special character sets are at
<https://docs.python.org/2/howto/regex.html>

RE: Escapes

Sometimes you want to literally match a metacharacter like “.”

- Example: “Mary runs. Sam and Susan don’t”
- The pattern `.S` will match twice: “Mary runs. Sam and Susan don’t”
- How can you match the literal full stop (period) after “runs”?
- Use `\`: `\.` matches the full stop itself: “Mary runs. Sam and Susan don’t”
- `\`: backslash is used to **escape** the metacharacters:
 - Match `.` as a literal: `a\.c` matches “a.c”
 - Match `\` as a literal: `a\\c` matches “a\c”
 - Match `[` as a literal: `[\[` matches “[“
- So `\` is also a metacharacter
- You can use `\` to escape metacharacters inside `[]`
- You can escape the name of a character class
 - Match any character that is a member of that character class
 - The class of decimal digits is named `d`; `\d` matches any decimal digit

RE: Alternatives

- `|`: "OR" operator, for alternatives
- Given two patterns P1 and P2, `P1 | P2` will match P1 or P2

```
>>> text = 'Not sure if her name is Mal or Mel. Do you remember?'
>>> name = re.findall(r'Mel|Mal', text)
>>> name
['Mal', 'Mel']
>>> |
```

RE: Anchors

- `^` and `$` : Anchoring
- `^` : must be start of string
 - `^from` will match from only at the start of the string, e.g., “from Melbourne”
 - `^from` will not match “I am from Melbourne”
- `$` : must be end of string
- What if you want to match literally the characters “`^`” or “`$`” ?
 - Escape with `\^` or `\$`
 - Or put it in character class `[$^]` (note special meaning if `^` is placed as the first character)

RE: Repeating

- `*` `+` `?` `{m,n}` : repeat a pattern
 - `*` : zero or more
 - `+` : one or more
 - `?` : zero or one

```
In [1]: import re
In [2]: text = 'gd, god, good, gooooooooood'
In [3]: re.findall(r'go*d', text)
Out[3]: ['gd', 'god', 'good', 'gooooooooood']
In [4]: re.findall(r'go+d', text)
Out[4]: ['god', 'good', 'gooooooooood']
In [5]: re.findall(r'go?d', text)
Out[5]: ['gd', 'god']
```

- `{m,n}` : at least m and at most n repetitions

```
In [8]: text = 'This is COMP20008, Week 2.'
In [9]: re.findall(r'\d{1,2}', text)
Out[9]: ['20', '00', '8', '2']
```

- Pattern search is greedy

```
In [8]: text = 'gogogoD'
In [9]: pattern = '[go]+'
In [10]: answer = re.findall(pattern, text)
In [13]: print(answer)
```

RE: Marked sub-expression and substitution

- **()**: just as in math notation, group patterns within the section
- Grouped patterns are captured and numbered
- You can specify the contents by back-references

```
In [15]: text = 'To Be Or Not To Be? That is the question.'
```

```
In [16]: re.findall(r'(.+) Or Not \1', text)
```

```
Out[16]: ['To Be']
```

```
In [18]: re.findall(r'(.+) Or Not \1.+ (question)', text)
```

```
Out[18]: [('To Be', 'question')]
```

RE: Lookahead assertions

- Specify what comes immediately after a match
 - `(?= )` : positive lookahead assertion. The enclosed expression must also be matched if the preceding pattern can be a match
 - `(?! )` : negative lookahead assertion. The enclosed expression must **not** match if the preceding pattern can be a match
- They are not part of the matched pattern
 - `\d+(?=AUD)` : matches amounts only if it IS followed by AUD
 - `\d+(?!AUD)` : matches amounts only if it IS NOT followed by AUD

Re: All metacharacters

The complete list of metacharacters

. ^ \$ * + ? { } [] \ | ()

More complex regular expression

- What do you think this pattern is for?

`[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+`

Lower or uppercase
letters or numbers etc.

One or
more

"@"
sign

Same as first
part

Literal
"

Again same
as first part

More complex regular expression

- What do you think this pattern is for?

`[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+`

Lower or uppercase
letters or numbers etc.

One or
more

"@"
sign

Same as first
part

Literal
"

Again same
as first part

- All email addresses!

Why is string matching useful?

Spelling correction, Typo

vs Word equivalence

vs Neologisms (= new words)

Spelling correction

- The problem: Given a substring (word) that does **not** appear in the dictionary, which entry from the dictionary was intended by the user?

- Common solution:

- Find all words 'close by' (= small letter-gram distance)
- Look at left and right words
- Select the close word that best fits the context window

“... proceed if you can see no ther option ... ← no otter option
“ ? no other option
 no here option

- Need a list of all the frequent word ngrams in the language
- Depends on the person who wrote the original string!

Neologisms

- A **Neologism** is a new word: *phat*, *ChatGPT*, *arvo*...
- **Example: Word Blending**
(forming novel words by “blending” two other words)
 - breakfast + lunch → brunch
 - fork + spoon → spork
 - Britain + exit → Brexit
- **Language changes continuously**
 - Social media is fertile ground for linguistic innovation
- **No simple solution:**
(You just have to extend the dictionary!)

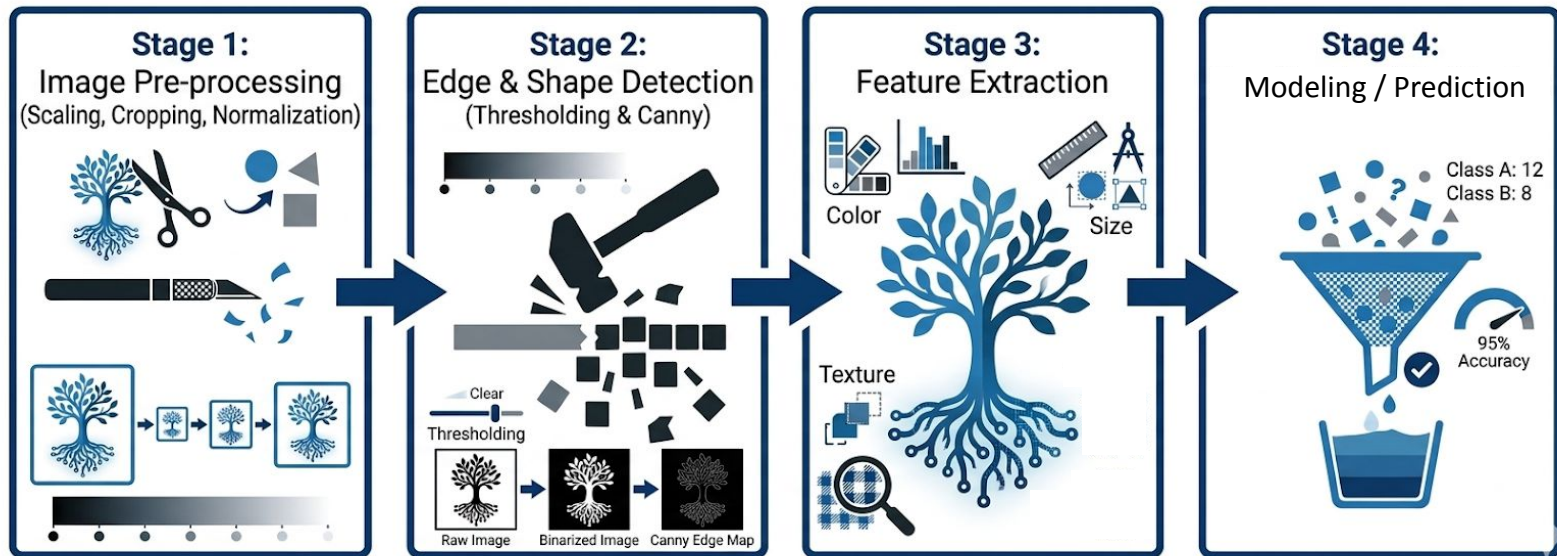


BRUNCH

Word equivalence

- Closely equivalent word forms
- American and British spellings
 - color = colour, defence = defense
- Street and place name conventions
 - boulevard|blvd|bd|bde|blv|bl|blvde|blvrd|boulavard|boul|bvd
 - apartment|apt|ap|aprt|aptmnt
 - village|vil|vge|vill|villag|villg|vlg|vlge|vllg

Image Processing Pipeline



You will learn more details in Machine Learning (COMP30027)

Week 4 Workshop & Lecture 3

Alignment

Workshop Goal

- Extract text from web / raw sources (scraping)
- Clean and preprocess text (tokenisation, stopwords, etc.)
- Identify patterns in text (regex)
- Perform basic text matching and comparison